

ABC458 E - Count 123 题解

题意概括

统计长度为 $x_1 + x_2 + x_3$ 的序列个数，要求：

- 恰好有 x_1 个 1、 x_2 个 2、 x_3 个 3；
- 相邻两个数的差的绝对值不超过 1。

答案对 998244353 取模。

解题思路

第一步：先把所有 2 固定下来

因为 1 和 3 不能相邻，所以可以先把 x_2 个 2 排成一行。

这时会形成 $x_2 + 1$ 个空隙：

- 第一个 2 前面一个空隙；
- 每两个相邻 2 之间一个空隙；
- 最后一个 2 后面一个空隙。

记空隙数为 $G = x_2 + 1$ 。

第二步：每个空隙里只能放三种情况

一个空隙里可以放：

- 什么都不放；
- 一段正长度的 1；
- 一段正长度的 3。

不能在同一个空隙里同时放 1 和 3，否则它们会相邻，违反条件。

所以单个空隙的生成函数是：

$$1 + (x + x^2 + x^3 + \dots) + (y + y^2 + y^3 + \dots)$$

也就是：

$$1 + \frac{x}{1-x} + \frac{y}{1-y} = \frac{1-xy}{(1-x)(1-y)}$$

于是总生成函数为：

$$\left(\frac{1 - xy}{(1 - x)(1 - y)}\right)^G$$

我们要求的就是 $x^{X_1} y^{X_3}$ 的系数。

第三步：把生成函数展开

先展开：

- $(1 - xy)^G = \sum (-1)^k \binom{G}{k} x^k y^k$
- $(1 - x)^{-G}$ 中 x^t 的系数是 $\binom{G + t - 1}{t}$
- $(1 - y)^{-G}$ 中 y^t 的系数同理

所以答案就是：

$$\sum_{k=0}^{\min(G, X_1, X_3)} (-1)^k \binom{G}{k} \binom{G + X_1 - k - 1}{X_1 - k} \binom{G + X_3 - k - 1}{X_3 - k}$$

只要预处理阶乘和逆阶乘，就能在线性时间内求完这段和式。

正确性说明

先固定 X_2 个 2 后，任意一个合法序列都可以唯一分解成 $G = X_2 + 1$ 个空隙：

- 每个空隙要么为空；
- 要么是一段连续的 1；
- 要么是一段连续的 3。

反过来，只要每个空隙按这三种方式之一填写，就一定得到一个相邻差绝对值不超过 1 的合法序列。

因此题目答案恰好等于上述生成函数中 $x^{X_1} y^{X_3}$ 的系数。

接下来对生成函数逐项展开：

- $(1 - xy)^G$ 决定有多少个空隙同时为 1 和 3 的“修正项”；
- $(1 - x)^{-G}$ 和 $(1 - y)^{-G}$ 分别统计把 X_1 个 1、 X_3 个 3 分配到 G 个空隙中的方式数。

把三部分相乘并取对应系数，就得到题解中的求和公式。

因此算法得到的答案与题目要求完全一致，算法正确。

复杂度

设 $M = \max(X_1 + X_2, X_2 + X_3)$ 。

- 预处理阶乘与逆阶乘： $O(M)$
- 枚举求和： $O(\min(X_1, X_2 + 1, X_3))$
- 空间复杂度： $O(M)$

参考实现

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;

const ll MOD = 998244353;

ll qpow(ll a, ll b){
    ll res = 1;
    while(b > 0){
        if(b & 1){
            res = res * a % MOD;
        }
        a = a * a % MOD;
        b >>= 1;
    }
    return res;
}

int main(){

    int x1, x2, x3;
    cin >> x1 >> x2 >> x3;

    int g = x2 + 1;
    int lim = max(x1 + x2, x2 + x3);

    vector<ll> fac(lim + 1), ifac(lim + 1);
    fac[0] = 1;
    for(int i=1; i<=lim; i++){
        fac[i] = fac[i - 1] * i % MOD;
    }

    ifac[lim] = qpow(fac[lim], MOD - 2);
    for(int i=lim; i>=1; i--){
        ifac[i - 1] = ifac[i] * i % MOD;
    }

    auto C = [&](int n, int r) -> ll {
        if(r < 0 || r > n){
```

```

        return 0;
    }
    return fac[n] * ifac[r] % MOD * ifac[n - r] % MOD;
};

ll ans = 0;
int up = min(g, min(x1, x3));
for(int k=0; k<=up; k++){
    ll cur = C(g, k);
    cur = cur * C(g + x1 - k - 1, x1 - k) % MOD;
    cur = cur * C(g + x3 - k - 1, x3 - k) % MOD;

    if(k & 1){
        ans = (ans - cur + MOD) % MOD;
    } else {
        ans = (ans + cur) % MOD;
    }
}

cout << ans;
return 0;
}

```